.

Pontificia Universidad Católica Madre y Maestra

Facultad de Ciencias e Ingeniería

Escuela de Ingeniería en Computación y Telecomunicaciones

Asignatura:

Algoritmos Clásicos

Práctica/Tarea:

Proyecto GPS

Estudiantes:

Eduardo Antonio Hernández Grullón (10159157)

Elián José Tejada Pérez (10159774)

Profesor:

Prof. Freddy Peña

Fecha de Entrega: 7/4/2026

I.Introducción:.....3
II Desarrollo:.....4
1. Estructura de datos.....4
2. Análisis de Complejidad de las Operaciones:.....4
2. Justificación y Complejidad de Algoritmos de Búsqueda.....5
2.1. Algoritmo de Dijkstra.....5
2.2. Algoritmo de Bellman-Ford.....5
2.3. Búsqueda en Anchura (BFS).....6
2.4. Búsqueda en Profundidad (DFS) y Rutas Alternativas.....6
2.5 Algoritmo Floyd-Warshall.....7
3. Funcionalidades del sistema.....7
3.1. Gestión de paradas y rutas.....7
3.2. Cálculo de rutas óptimas.....8
3.3. Generación de rutas alternativas.....8
3.4. Persistencia y gestión de datos.....9
4. Decisiones de Diseño e Interfaz de Usuario.....9
4.1. Filosofía de Diseño de Interfaz (UI).....9
4.2. Identidad Visual y Paleta de Colores.....9
4.3. Consistencia entre Diseño y Código.....10
III. Conclusión.....11

I.Introducción:

El presente informe documenta el desarrollo de Urbana GPS, el proyecto final de la asignatura Algoritmos Clásicos y Estructuras de Datos. El sistema ha sido diseñado como una solución integral para la gestión y optimización de rutas de transporte urbano, permitiendo a los usuarios planificar trayectos eficientes dentro de una red metropolitana.

El objetivo principal de la aplicación es calcular la ruta óptima entre dos paradas seleccionadas por el usuario, evaluando múltiples criterios de peso como el tiempo estimado de viaje, la distancia física, los costos de pasaje y la cantidad de transbordos necesarios. Para alcanzar este nivel de precisión, se implementó un grafo dirigido utilizando listas de adyacencia como estructura de datos medular, garantizando la eficiencia en la representación de redes viales densas.

Sobre esta estructura, se aplicaron y compararon algoritmos de búsqueda y optimización de vanguardia, tales como Dijkstra y Bellman-Ford, además de emplear búsquedas en profundidad (DFS) para la generación de rutas alternativas. El sistema se complementa con una interfaz gráfica interactiva desarrollada en JavaFX y un módulo de persistencia basado en archivos JSON, lo que asegura la disponibilidad de la información entre diferentes sesiones de uso.

Este documento describe detalladamente la arquitectura de datos seleccionada, el análisis de complejidad asintótica de los algoritmos implementados, las decisiones técnicas de diseño y las pruebas de validación que confirman la robustez del sistema.

II Desarrollo:

1. Estructura de datos

La arquitectura del sistema se fundamenta en un Grafo Dirigido Ponderado, una elección técnica superior para modelar redes de transporte urbano donde el sentido de las vías y los costos asociados son variables críticas. Para su implementación, se optó por listas de adyacencia gestionadas mediante la estructura “Map<Parada”, “List<Ruta>>”. Esta decisión garantiza una eficiencia espacial de O(V + E), optimizando el uso de memoria en grafos dispersos como los de una red vial metropolitana.

El modelo de datos se desglosa en tres componentes esenciales que aseguran la modularidad del código:

- Parada (Nodo): Representa cada estación o punto de control en el mapa. Encapsula metadatos como el identificador único (ID), nombre descriptivo, localización y coordenadas cartesianas (X, Y) para el renderizado interactivo en la interfaz gráfica.
- Ruta (Arista): Modela la conexión dirigida entre dos paradas. Cada objeto carga con el peso multidimensional necesario para la optimización: tiempo estimado en minutos, distancia física en kilómetros, costo del pasaje y el indicador de transbordo.
- GrafoTransporte: Actúa como el motor de la estructura, utilizando un HashMap para mapear cada objeto Parada con su respectiva lista de rutas salientes, facilitando el recorrido eficiente del grafo.

2. Análisis de Complejidad de las Operaciones:

En cuanto al rendimiento operativo sobre esta estructura, se han determinado las siguientes complejidades asintóticas en función del número de paradas (V) y rutas (E):

La inserción de una nueva parada se realiza en tiempo constante, O(1), al utilizar el método “putIfAbsent()” del HashMap. De igual manera, la inserción de una ruta presenta una complejidad de O(1), ya que implica localizar la parada origen por su ID y añadir el elemento al final de una ArrayList.

Por otro lado, la eliminación de una parada es la operación más costosa, con una complejidad de O(V + E), debido a que el sistema debe remover el nodo del mapa y realizar un barrido integral para eliminar cualquier referencia a dicha parada en las rutas de destino de otros nodos. Finalmente, la consulta de vecinos se ejecuta en O(1) gracias al acceso directo por clave en el mapa, mientras que la verificación de existencia de una arista requiere un tiempo de O(E_out), proporcional al grado de salida del nodo origen (E_out es la cantidad de rutas que salen de una parada en particular).

2. Justificación y Complejidad de Algoritmos de Búsqueda

En esta sección se detallan los algoritmos implementados en el sistema Urbana GPS, justificando su selección técnica y analizando su eficiencia mediante notaciones asintóticas de tiempo (T) y espacio (S).

2.1. Algoritmo de Dijkstra

Es el motor principal para el cálculo de la ruta óptima bajo criterios de tiempo, distancia o costo. Su implementación utiliza una PriorityQueue para optimizar la selección del nodo con la menor distancia tentativa en cada iteración.

- Complejidad Temporal: O((V + E) log V).
    - Justificación: El término V log V proviene de las extracciones del valor mínimo de la cola de prioridad (una por cada nodo), mientras que el término E log V se deriva de las actualizaciones de prioridad (relajación de aristas) dentro del heap binario.
- Complejidad Espacial: O(V + E).
    - Justificación: Se requiere almacenar el mapa de distancias mínimas y los punteros a los nodos padres O(V), además de la estructura de la lista de adyacencia que contiene todas las conexiones O(V + E).

2.2. Algoritmo de Bellman-Ford

Se incluyó para garantizar la robustez del sistema ante posibles escenarios de tarifas negativas (descuentos promocionales) que Dijkstra no puede procesar.

- Complejidad Temporal: O(V x E).
    - Justificación: El algoritmo realiza |V| - 1 iteraciones sobre el conjunto total de aristas |E| para asegurar que la información de

distancia mínima se propague por todo el grafo, incluso en los trayectos con mayor número de conexiones.

- Complejidad Espacial: O(V).
    - Justificación: A diferencia de Dijkstra, solo necesita mantener arreglos lineales para costos y padres, sin requerir una estructura de datos compleja como una cola de prioridad.

2.3. Búsqueda en Anchura (BFS)

El sistema emplea BFS para el cálculo de rutas basadas en el criterio de "Mínimos Transbordos".

- Complejidad Temporal: O(V + E).
    - Justificación: El algoritmo explora el grafo por niveles, visitando cada nodo y cada arista exactamente una vez para encontrar el camino con el menor número de saltos topológicos.
- Complejidad Espacial: O(V).
    - Justificación: Requiere memoria para el conjunto de nodos visitados y la cola de procesamiento de niveles.

2.4. Búsqueda en Profundidad (DFS) y Rutas Alternativas

El sistema utiliza DFS de forma recursiva para cumplir con el requisito de mostrar opciones secundarias mediante una exploración exhaustiva de caminos simples.

- Complejidad Temporal: O(V + E).
    - Justificación: El método “encontrarTodosLosCaminos” recorre la estructura de adyacencia visitando sistemáticamente los nodos para construir el árbol de trayectos posibles entre origen y destino.
- Complejidad Espacial: O(V).
    - Justificación: La memoria utilizada es proporcional a la profundidad de la pila de recursión y al conjunto de nodos marcados durante la exploración.

2.5 Algoritmo Floyd-Warshall

El sistema emplea este algoritmo para la generación de la "Matriz de Caminos", permitiendo visualizar y calcular el costo mínimo global (ya sea en tiempo, distancia o tarifa) entre todas las paradas de la red de transporte público de forma simultánea. Es una herramienta de auditoría estratégica ideal para evaluar la conectividad total del sistema a gran escala.

- Complejidad Temporal: $O(V^3)$
    - Justificación: Su estructura lógica se basa en la ejecución de tres bucles iterativos anidados que recorren la totalidad de los vértices (V) del grafo. Este proceso sistemático evalúa cada nodo existente como un punto intermedio potencial para verificar si optimiza el costo del trayecto entre cualquier par de paradas origen-destino.
- Complejidad Espacial: $O(V^2)$
    - Justificación: A diferencia de las búsquedas simples, su ejecución exige la creación de una matriz de distancias bidimensional de tamaño V x V en la memoria principal. Esta estructura es estrictamente necesaria para mantener y actualizar el registro constante de los costos mínimos entre todos los pares de nodos posibles simultáneamente.

3. Funcionalidades del sistema

El sistema Urbana GPS ha sido desarrollado para ofrecer una solución integral a la problemática del transporte urbano, permitiendo una interacción dinámica entre el usuario y la red de paradas. Las funcionalidades principales se desglosan a continuación:

3.1. Gestión de paradas y rutas

El sistema proporciona un módulo administrativo completo (CRUD) para la configuración de la red de transporte:

- Administración de Nodos: El usuario tiene la capacidad de agregar, modificar y eliminar paradas, asignándoles un identificador, nombre y coordenadas espaciales para el mapa interactivo.

- Control de Aristas: Es posible establecer conexiones dirigidas entre paradas, definiendo los pesos de tiempo, distancia y costo que serán utilizados por los algoritmos de optimización.
- Validaciones de Integridad: Se implementaron controles automáticos que impiden el ingreso de valores negativos, la creación de rutas duplicadas o la referencia a paradas inexistentes, asegurando que el grafo en memoria sea siempre coherente.

3.2. Cálculo de rutas óptimas

La funcionalidad medular del sistema permite determinar el trayecto más eficiente entre dos puntos geográficos:

- Selección Interactiva: El usuario selecciona el origen y el destino, ya sea mediante formularios de texto o interactuando directamente con los nodos representados en el mapa de JavaFX.
- Multicriterio de Optimización: El motor de cálculo permite ajustar la búsqueda según el factor de interés: el trayecto más rápido (tiempo), el más corto (distancia), el más económico (costo) o el que requiera menos cambios de unidad (transbordos).
- Métricas en Tiempo Real: Tras cada cálculo, el sistema despliega un desglose detallado de los valores acumulados del trayecto, facilitando la toma de decisiones del usuario.

3.3. Generación de rutas alternativas

Para robustecer la experiencia de navegación, el sistema ofrece una comparativa de trayectos mediante un análisis dual:

- Exploración Topológica: A través de una búsqueda exhaustiva (DFS), se identifican todos los caminos simples entre los puntos seleccionados.
- Visualización Dual: El sistema renderiza simultáneamente la Ruta Óptima (Verde) y la Ruta Alternativa (Oro/Amarilla), permitiendo observar visualmente las diferencias geográficas entre la mejor opción y la segunda opción más viable.

3.4. Persistencia y gestión de datos

La integridad de la red de transporte está garantizada por un módulo de almacenamiento local:

- Almacenamiento JSON: Se utiliza la librería Jackson para serializar el estado del grafo en un archivo físico, permitiendo que todas las modificaciones realizadas persistan entre diferentes sesiones de uso.
- Sincronización Automática: Al iniciar la aplicación, el sistema reconstruye el grafo en memoria a partir del archivo de datos, asegurando que la interfaz gráfica refleje siempre la configuración más reciente de la red.

4. Decisiones de Diseño e Interfaz de Usuario

El desarrollo de Urbana GPS priorizó la usabilidad y la claridad visual, siguiendo los criterios de evaluación de interfaces modernas. Las decisiones estéticas y funcionales se fundamentaron en dos pilares:

4.1. Filosofía de Diseño de Interfaz (UI)

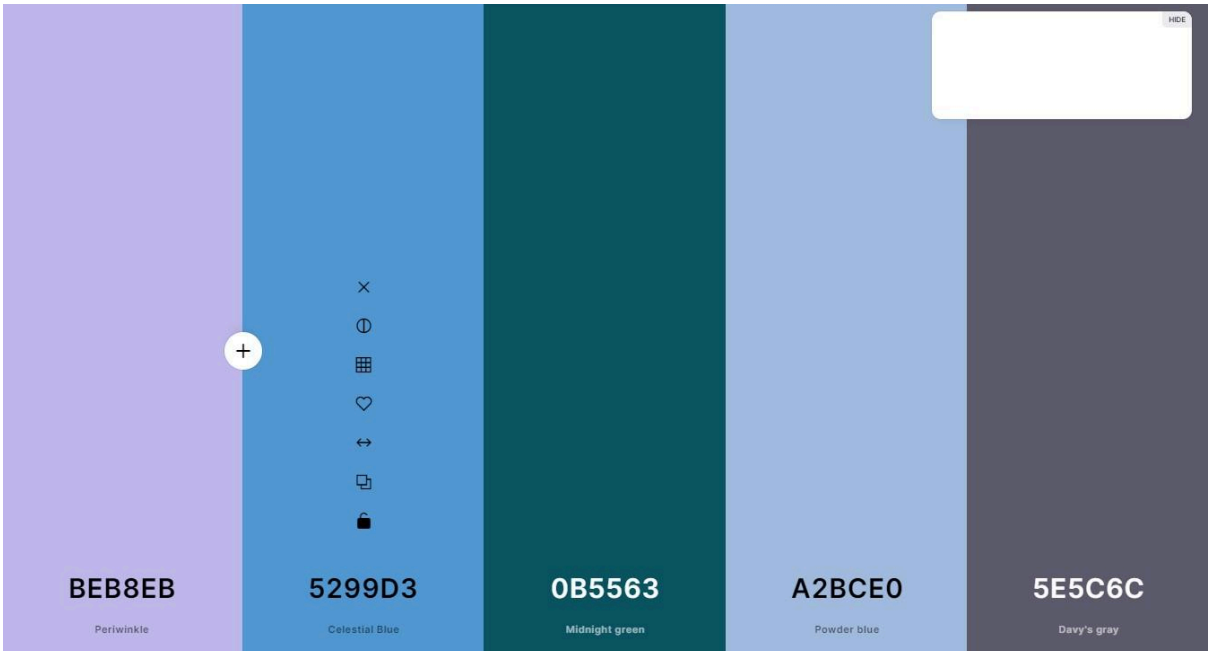
Se optó por un estilo de diseño basado en mosaicos para el menú principal, inspirado en sistemas de gestión de activos y monitoreo GPS.

- Interactividad: Este diseño permite una navegación intuitiva mediante botones de gran formato que facilitan la interacción táctil o mediante puntero, reduciendo la carga cognitiva del usuario.
- Organización: Cada funcionalidad (Mapa, Gestión de Paradas, Configuración) está aislada visualmente, lo que previene errores de navegación y mejora la eficiencia operativa del sistema.

4.2. Identidad Visual y Paleta de Colores

Para garantizar una estética profesional y coherente, se definió una paleta de colores técnica y sobria.

- Color Primario (Midnight Green - #0B5563): Seleccionado como el tono principal para la representación de nodos (paradas) en el mapa y elementos de cabecera, proporcionando un contraste óptimo con el fondo claro del canvas.
- Jerarquía Visual: Se utilizaron tonos complementarios como el Celestial Blue (#5299D3) para elementos informativos y el Davy's Gray (#5E5C6C) para textos secundarios, asegurando una lectura clara de las métricas de tiempo y costo.



4.3. Consistencia entre Diseño y Código

La implementación en JavaFX refleja fielmente estas decisiones. En la clase “TransporteVisual.java”, se hardcodeó el valor hexadecimal #0B5563 para el relleno de los óvalos que representan las paradas, asegurando que la ejecución del software sea una extensión exacta del manual de estilo definido.

III. Conclusión

El desarrollo del sistema Urbana GPS ha permitido integrar de manera práctica los conceptos fundamentales de la asignatura Algoritmos Clásicos y Estructuras de Datos, demostrando que la selección técnica de la arquitectura es el pilar de cualquier solución de software exitosa. La implementación de un grafo dirigido mediante listas de adyacencia resultó ser la estructura más adecuada para representar redes de transporte urbano, garantizando una eficiencia espacial de O(V + E) y permitiendo una manipulación dinámica de la red sin penalizar el rendimiento del sistema. Asimismo, la combinación estratégica de los algoritmos de Dijkstra y Bellman-Ford permitió abordar el problema de la optimización desde perspectivas complementarias; mientras el primero aseguró la rapidez necesaria para consultas en tiempo real, el segundo aportó la robustez requerida para manejar escenarios complejos con pesos negativos o incentivos tarifarios.

Por otro lado, la innovación introducida mediante el uso de una búsqueda en profundidad (DFS) para la generación de rutas alternativas permitió superar las limitaciones de los algoritmos de optimización tradicionales que retornan una solución única. Al identificar la topología completa de caminos y aplicar un ordenamiento multicriterio, el sistema logró ofrecer una comparativa de trayectos de alto valor para el usuario final. Finalmente, las decisiones de diseño basadas en una paleta de colores técnica y una interfaz interactiva en JavaFX confirmaron que la claridad visual y la usabilidad son componentes críticos que potencian la precisión de los datos calculados.

En definitiva, el proyecto no solo cumple con los requerimientos académicos establecidos, sino que se posiciona como una herramienta robusta, escalable y persistente, capaz de resolver problemas reales de optimización en el ámbito de la movilidad urbana.